EXPERIMENT NO: 6

Student Name: Aaditya Anand
UID: 23BCS10446
Branch: BE-CSE
Section/Group: KRG-3A
Semester: 6th
Date of Performance: 26/02/2026
Subject Name: System Design
Subject Code: 23CSH-314

Aim

To design a highly scalable video streaming platform similar to YouTube that supports adaptive bitrate streaming (HLS/DASH), dynamic manifest handling, live streaming with sliding window playback, and real-time engagement features (likes and comments) with high availability and low latency.

Objectives

- Design a scalable YouTube-like video streaming architecture supporting Video-on-Demand and live streaming.
- Implement a video upload pipeline with encoding, transcoding into multiple resolutions, and segment generation.
- Apply adaptive streaming using manifest files (MPD) and segmented video delivery through CDN.
- Develop scalable interaction systems for likes, comments, and user engagement.
- Use event-driven processing (Kafka) for handling watch events and analytics.
- Ensure high availability, scalability, and fault tolerance using distributed services and caching.

Procedure

1. Designed a scalable video streaming architecture including API gateway, microservices, and CDN distribution.
2. Implemented a video upload pipeline with encoding, transcoding, and multi-resolution segment generation.
3. Designed adaptive streaming using manifest files and segmented video delivery.
4. Implemented scalable like and comment systems using Redis counters and database storage.
5. Integrated event streaming (Kafka) for processing watch events and engagement data.
6. Designed analytics and recommendation pipeline using feature extraction and ML ranking models.
7. Ensured efficient storage using object storage for video segments, thumbnails, and metadata.

Functional Requirements

1. Users should be able to upload and stream videos.
2. Videos should be processed and available in multiple resolutions (240p–4K).
3. The system should support adaptive bitrate streaming using manifest files.
4. Users should be able to like and comment on videos.
5. The platform should support live streaming with real-time interaction.
6. The system should provide scalable storage and delivery of video content.

Core Entities of the System

• Users
• Videos
• Video Metadata
• Video Segments
• Manifest Files
• Comments
• Likes / Interactions
• Subscriptions
• Playlists
• Watch Events / View History

API Design

1. Authentication API

• Register: **POST /api/v1/auth/register**
• Login: **POST /api/v1/auth/login**

2. Video API

• Initialize Upload: **POST /api/v1/videos/upload/init**
• Get Video Metadata: **GET /api/v1/videos/{video_id}**
• Get Streaming Manifest: **GET /api/v1/videos/{video_id}/manifest**

3. Feed & Search API

• Get Personalized Feed: **GET /api/v1/feed** (Pagination Supported)
• Search Videos: **GET /api/v1/search?q={query}**

4. Interaction API

• Like Video: **POST /api/v1/videos/{video_id}/like**
• Add Comment: **POST /api/v1/videos/{video_id}/comments**
• Get Comments: **GET /api/v1/videos/{video_id}/comments**

5. Subscription API

• Subscribe Channel: **POST /api/v1/channels/{channel_id}/subscribe**
• Unsubscribe Channel: **DELETE /api/v1/channels/{channel_id}/subscribe**

6. Playlist API

• Create Playlist: **POST /api/v1/playlists**
• Add Video to Playlist: **POST /api/v1/playlists/{playlist_id}/videos**

Non-Functional Requirements

1. The system must support **high scalability**, handling millions of concurrent users, uploads, and video views.
2. The platform should maintain **high availability** with distributed services and no single point of failure.
3. The system should use **eventual consistency** for engagement metrics such as views, likes, and comments to reduce database load.
4. Video delivery latency must be minimized using **CDN edge servers and caching mechanisms**.
5. The video processing pipeline must be **horizontally scalable** to process large volumes of uploaded videos.
6. The system should ensure **fault tolerance and reliability** using distributed storage and event-driven processing.

High Level Design (HLD)
Streaming Architecture (DASH)
The platform uses **Dynamic Adaptive Streaming over HTTP (DASH)** for efficient video streaming.

Video Upload and Processing Pipeline

Flow:
Client → API Gateway → Upload Service → Raw Video Storage
After upload:
Raw Video Storage → Video Processing Workers → Encoder → Transcoder
Each uploaded video is converted into **multiple resolutions**:

- 240p
- 360p
- 480p
- 720p
- 1080p
- 4K

Each resolution is divided into **small segments (2–6 seconds)** to support adaptive streaming.

Video Chunking and Manifest Generation
Flow:
Transcoded Video → Video Chunker → Manifest Generator (MPD)
Manifest File (.mpd) contains:

- Available video qualities
- URLs of video segments
- Streaming metadata

This enables the video player to dynamically select the most suitable bitrate.

Adaptive Bitrate Streaming
The video player continuously monitors network bandwidth.
Process:

- Player measures available bandwidth
- Selects appropriate resolution
- Downgrades quality when bandwidth decreases
- Upgrades quality when bandwidth improves
- Switching occurs at segment boundaries

This ensures **smooth playback without buffering**.

Content Delivery
Processed video segments are stored in **Object Storage**.
Delivery flow:
Object Storage → Origin Server → CDN Edge Servers → Video Player
CDN caching allows global users to access videos with **low latency and high throughput**.

Event Processing and Analytics

User watch events are processed using an **event-driven architecture**.

Flow:

Video Player → Watch Events → Kafka Stream → Analytics Processing

Analytics results are stored in the **Analytics Data Warehouse** and used for:

- video popularity tracking
- engagement analysis
- recommendation models

Like System Design

Likes are **high-frequency operations**.

Flow:

Client → API Gateway → Like Service

Processing:

- Redis counter increment for fast updates
- Event sent to Kafka stream
- Database updated asynchronously

Duplicate prevention:

UNIQUE(user_id, video_id)

or

Redis SETNX

This approach ensures **high performance and scalability**.

Comment System Design

Comments are stored in a **NoSQL database** for scalability.

Database structure:

Partition Key → video_id

Sort Key → timestamp

Advantages:

- horizontal scalability
- efficient pagination
- fast retrieval of comments

Replies are structured using:

comment_id

parent_id

thread_root_id

Real-Time Chat (Live Streams)

Live chat uses **WebSocket communication**.

Flow:

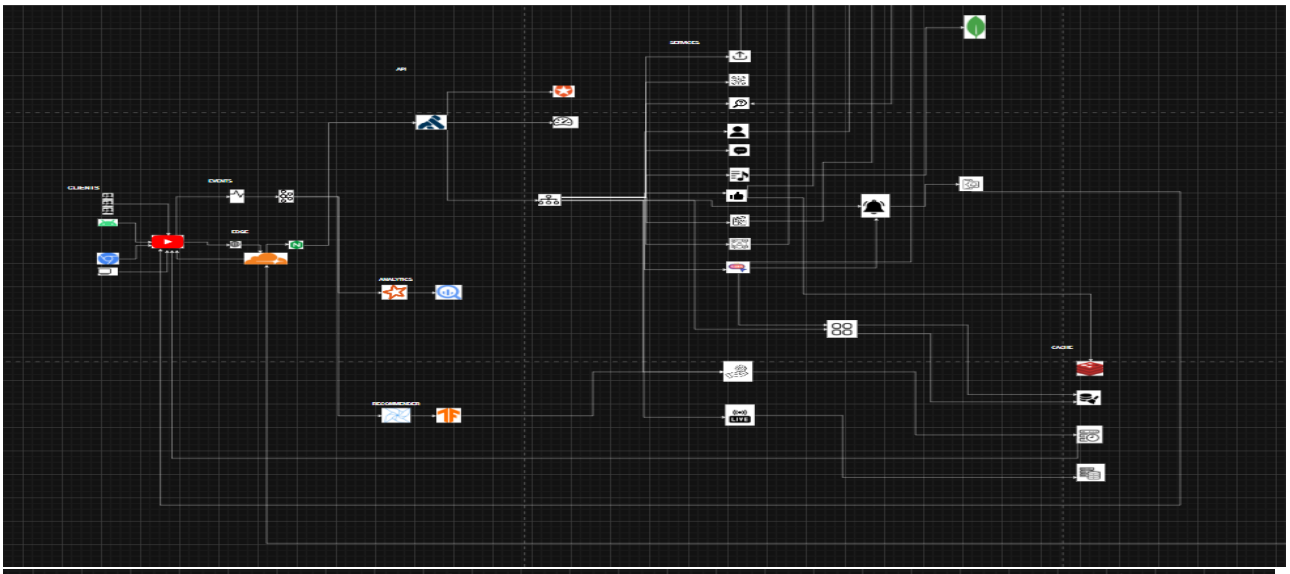
Client → WebSocket Gateway → Chat Server

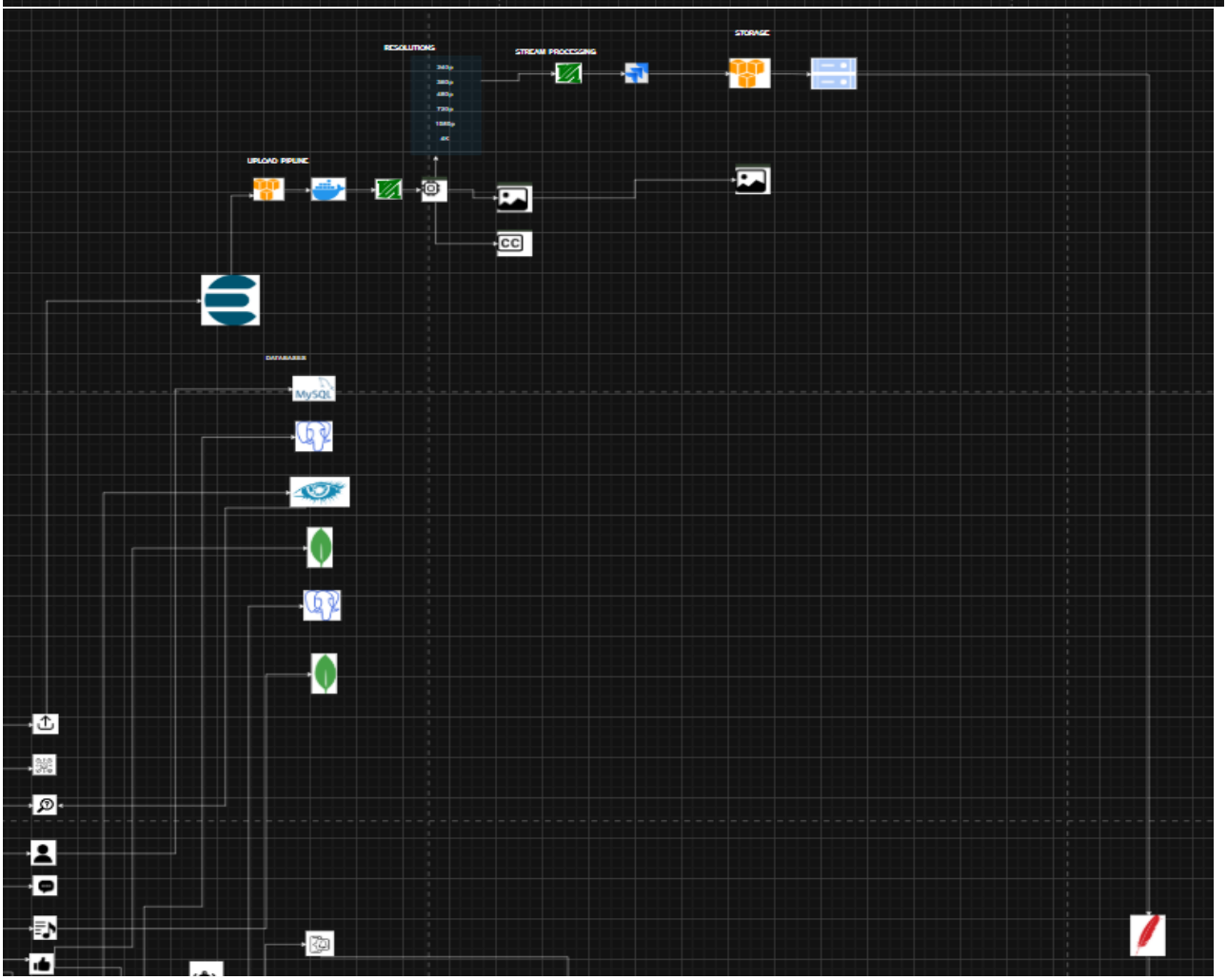
Chat Server → Pub/Sub → Connected Clients

Each live stream has a **separate chat shard** to handle high message volumes.

Chat persistence strategy:

- temporary in-memory storage for live chat
- archived storage after the stream ends


CLIENTS
CDNS
CDNS
API
STORAGES
ANALYTICS
RECOMMENDER
CACHE


RESOLUTIONS
STREAM PROCESSING
STORAGE
UPLOAD PIPELINE
DATABASES
MySQL

Outcome

• Designed a scalable YouTube-like architecture supporting adaptive video streaming and distributed microservices.
• Implemented a video processing pipeline including encoding, transcoding, and segment generation.
• Integrated CDN and object storage for efficient video delivery.
• Built scalable like and comment systems for user interactions.
• Applied event-driven processing using streaming and analytics components.